Numerical Analysis (MAC F241)

Assignment

Ayush Kumar Singh

2024ADPS0644H

GitHub Repository

April 2026

Contents

Problem 1 **2**

Problem Statement . . . 2

Piecewise Linear Interpolant and Error Estimates . . . 2

Computation of Required Number of Subintervals . . . 3

MATLAB code for Piecewise Linear Interpolation . . . 4

Results for $f(x) = \frac{1}{1+x^2}$ . . . 6

Results for $f(x) = x^{\frac{1}{2}}$ . . . 8

Problem 2 **10**

Problem Statement . . . 10

Piecewise Quadratic Interpolant and Error Estimates . . . 10

Computation of Required Number of Subintervals . . . 11

MATLAB code for Piecewise Quadratic Interpolation . . . 12

Results for $f(x) = \sin(x)$ . . . 14

Results for $f(x) = x^{\frac{1}{2}}$ . . . 16

Problem 3 **19**

Problem Statement . . . 19

Designing an Algorithm for Adaptive Linear Approximation . . . 19

Implementation Details . . . 20

MATLAB Implementations for all Required Functions . . . 21

Results and Comparison with Other Methods . . . 24

- Results and Plots . . . 24
- Comparison with Piecewise Linear and Quadratic Approximations . . . 25
- Observations . . . 25

Problem 1

Problem Statement

Write a MATLAB code for piecewise linear Lagrange approximation. Compute the approximation to the following functions accurate to within ε.

- $f(x) = \frac{1}{1+x^2}$ in $[-5, 5]$; $\varepsilon = 0.000005$
- $f(x) = x^{\frac{1}{2}}$ in $[0, 1]$; $\varepsilon = 0.00005$

Show $f(x)$ and its approximation graphically and find the computational time.

Piecewise Linear Interpolant and Error Estimates

Let $a = x_0 < x_1 < \cdots < x_n = b$ be a partition of $[a, b]$. The **piecewise linear interpolant** $s(x)$ is defined by linear functions on each subinterval $[x_i, x_{i+1}]$:

$$s_i(x) = a_i + b_i(x - x_i), \quad i = 0, 1, \ldots, n-1,$$

where the interpolation conditions $s(x_i) = f(x_i)$, $s(x_{i+1}) = f(x_{i+1})$ give

$$\boxed{a_i = f(x_i)}, \quad \boxed{b_i = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}}.$$

Thus, $s(x)$ connects consecutive data points with straight lines.

Error Estimate: If $f \in C^2[a, b]$, the interpolation error satisfies

$$\max_{x \in [x_i, x_{i+1}]} |f(x) - s_i(x)| \leq \frac{1}{8} h_i^2 \max_{x \in [x_i, x_{i+1}]} |f''(x)|,$$

where $h_i = x_{i+1} - x_i$.

Taking the maximum over all subintervals:

$$\boxed{\max_{x \in [a,b]} |f(x) - s(x)| \leq \frac{1}{8} h^2 \max_{x \in [a,b]} |f''(x)|}$$

with $h = \max_i h_i$.

Uniform Partition Case If the nodes are equally spaced, $h = \frac{b-a}{n}$, then

$$\boxed{|e| \leq \frac{(b-a)^2}{8n^2} \max_{x \in [a,b]} |f''(x)|}$$

To guarantee $|e| < \varepsilon$, it suffices that

$$\boxed{n > \frac{b-a}{2} \sqrt{\frac{1}{2\varepsilon} \max_{x \in [a,b]} |f''(x)|}}$$

Computation of Required Number of Subintervals

We determine the required number of subintervals n using the error bound

$$n > \frac{b-a}{2}\sqrt{\frac{1}{2\varepsilon}\max_{x\in[a,b]}|f''(x)|}.$$

- **(a)** $f(x) = \frac{1}{1+x^2}$ on $[-5,5]$, with $\varepsilon = 5\times 10^{-6}$

$$f''(x) = \frac{6x^2-2}{(1+x^2)^3}, \quad \max_{[-5,5]}|f''(x)| = 2.$$

Substituting,

$$n > \frac{10}{2}\sqrt{\frac{1}{2\cdot 5\times 10^{-6}}\cdot 2} \approx 2236.07.$$

$$\boxed{n = 2237}$$

- **(b)** $f(x) = \sqrt{x}$ on $[0,1]$, with $\varepsilon = 5\times 10^{-5}$

$$f''(x) = -\frac{1}{4}x^{-3/2},$$

which is unbounded at $x=0$, so the standard error bound is not directly applicable.

Restricting to $[10^{-4},1]$,

$$\max|f''(x)| = \frac{1}{4(10^{-4})^{3/2}} = 2.5\times 10^5.$$

Substituting,

$$n > \frac{(1-10^{-4})}{2}\sqrt{\frac{1}{2\cdot 5\times 10^{-5}}\cdot 2.5\times 10^5} \approx 24997.5.$$

$$\boxed{n = 24998}$$

Note: This guarantees accuracy only on $[10^{-4},1]$. The error near $x=0$ may be large due to the singularity.

MATLAB code for Piecewise Linear Interpolation

We use the following function to build and plot **Piecewise Linear Approximations** :

```
function [] = Problem_1(f, a, b, n)

    tic;
    h = (b - a) / n; % Step Size

    % Choosing Uniformly Spaces n+1 points in the interval [a,b]
    x = linspace(a, b, n+1);

    % Calculate functional values at these points = ai's
    ai = f(x);

    % Initializing array for bi's
    bi = zeros(1, n);

    % Calculating bi's
    for i = 1 : n
        bi(i) = (ai(i+1) - ai(i)) / h;
    end

    toc;

    % Plotting
    figure;
    hold on;

    % Choosing Dense points (say 10000)
    test_points = linspace(a, b, 10000);
    functional_value = f(test_points);

    % Approx values
    approx_value = zeros(size(test_points));

    for j = 1:length(test_points)

        % Finding the subinterval
        i = floor((test_points(j) - a)/h) + 1;
        if i > n
            i = n;
        end

```

```
        % Piecewise linear formula
        approx_value(j) = ai(i) + bi(i)*(test_points(j) - x(i));
    end

    % Plot comparison
    plot(test_points, functional_value, 'b', 'LineWidth', 2);
    plot(test_points, approx_value, 'r--', 'LineWidth', 2);

    legend('Actual f(x)', 'Piecewise Linear Approx');
    title('Piecewise Linear Interpolation');
    xlabel('x'); ylabel('y');
    grid on;

    % Error
    max_error = max(abs(functional_value - approx_value));
    disp(['Maximum Error = ', num2str(max_error)]);

    % Error plot (pointwise error)
    error_values = abs(functional_value - approx_value);

    figure;
    plot(test_points, error_values, 'k', 'LineWidth', 2);

    title('Pointwise Error |f(x) - s(x)|');
    xlabel('x'); ylabel('Error');
    grid on;

    % Plotting Relative error
    rel_error = (error_values ./ max(abs(functional_value), 1e-12))
       *100;

    figure;
    plot(test_points, rel_error, 'r', 'LineWidth', 2);

    title('Relative Error |f(x) - s(x)| / |f(x)|');
    xlabel('x'); ylabel('Relative Error');
    grid on;

end
```

Function call parameters for $f(x) = \frac{1}{(1+x^2)}$ are as follows :

```
octave:1> f = @(x) 1./(1+x.^2);
octave:2> a = -5;
```

```
octave:3> b = 5;
octave:4> n = 2237;
octave:5> Problem_1(f, a, b, n);
```

Function call parameters for $f(x) = x^{\frac{1}{2}}$ are as follows :

```
octave:1> f = @(x) sqrt(x);
octave:2> a = 0;
octave:3> b = 1;
octave:4> n = 24998;
octave:5> Problem_1(f, a, b, n);
```

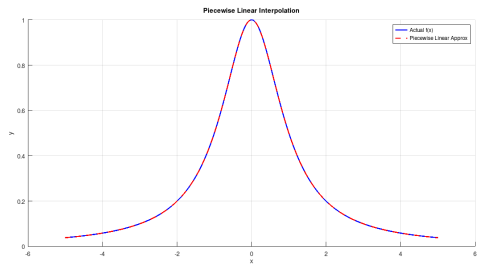
Results for $f(x) = \frac{1}{1+x^2}$

For $f(x) = \frac{1}{1+x^2}$, the numerical results are:

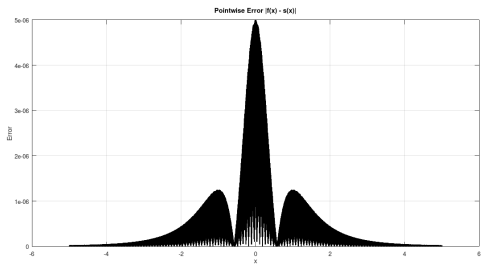
```
Elapsed time is 0.0243161 seconds.
Maximum Error = 4.9943e-06
```

The elapsed time excludes plot generation.

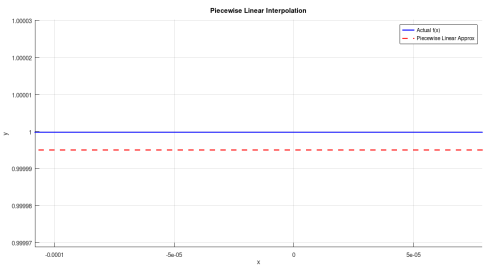
Plots



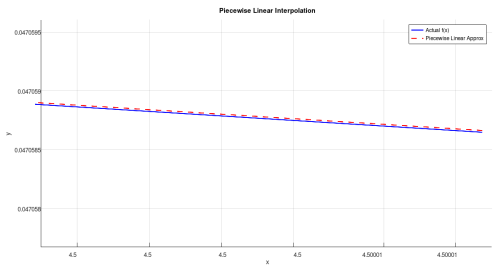
(a) Overall approximation on $[-5, 5]$



(b) Pointwise error



(c) Behaviour near $x = 0$



(d) Behaviour near $x = 4.5$

Figure 1: Approximation and error visualization

Error Analysis

x	Actual	Approx	Abs Error	Rel Error (%)
-1	0.5	0.5000008	7.99×10^{-7}	0.00016
-0.5	0.8	0.7999994	5.78×10^{-7}	0.000072
-0.25	0.9411765	0.9411732	3.31×10^{-6}	0.000351
-0.125	0.9846154	0.9846109	4.52×10^{-6}	0.000459
0	1	0.999995	4.99×10^{-6}	0.0005
0.125	0.9846154	0.9846109	4.52×10^{-6}	0.000459
0.25	0.9411765	0.9411732	3.31×10^{-6}	0.000351
0.5	0.8	0.7999994	5.78×10^{-7}	0.000072
1	0.5	0.5000008	7.99×10^{-7}	0.00016

Table 1: Sample error values

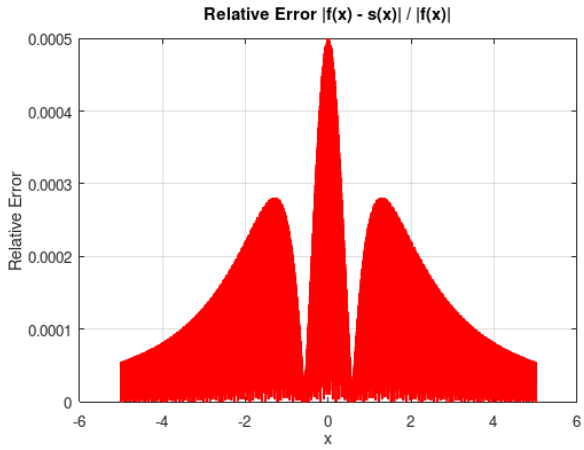


Figure 2: Relative error over $[-5, 5]$

Observations

1. The approximation is accurate since $f''(x)$ is bounded, though a large $n = 2237$ is required.

2. Errors are larger near $x = 0$ (higher curvature) and smaller near the endpoints.

3. This matches the theoretical bound

$$|f(x) - s(x)| \leq \frac{1}{8} h^2 \max |f''(x)|.$$

4. The interpolant captures the overall shape but struggles in high-curvature regions unless h is small.

5. Uniform partitioning is suboptimal; adaptive refinement near $x = 0$ would improve efficiency.

6. The observed maximum error satisfies the prescribed tolerance, validating the theory.

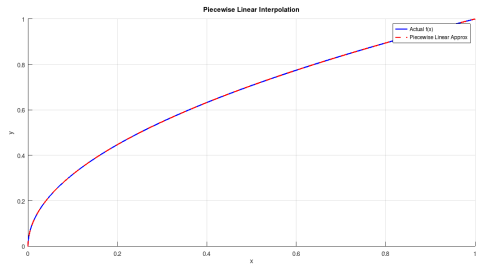
Results for $f(x) = x^{\frac{1}{2}}$

For $f(x) = x^{\frac{1}{2}}$, the numerical results are:

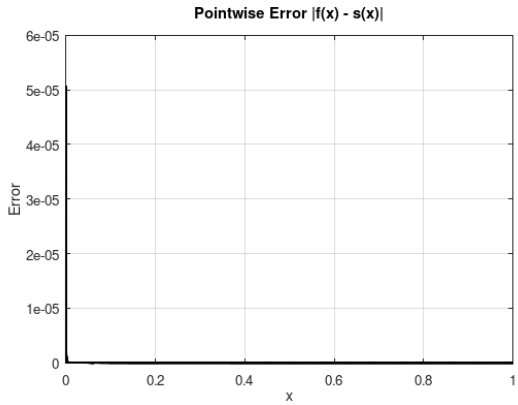
```
Elapsed time is 0.312404 seconds.
Maximum Error = 5.064e-05
```

The elapsed time excludes plot generation.

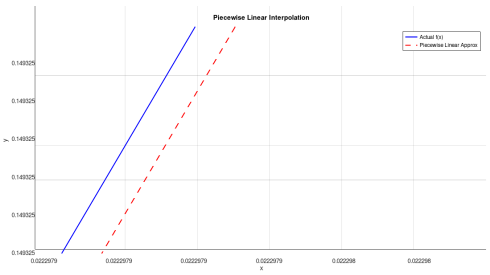
Plots



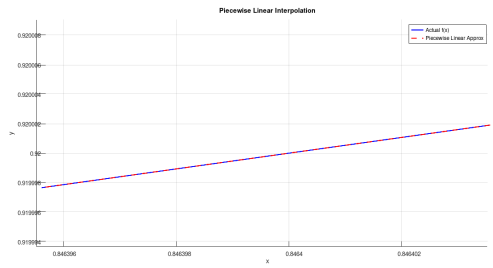
(a) Overall approximation on $[10^{-4}, 1]$



(b) Pointwise error



(c) Behaviour near $x = 0$



(d) Behaviour near $x = 1$

Figure 3: Approximation and error visualization

Error Analysis

x	Actual	Approx	Abs Error	Rel Error (%)
1	1	1	0	0
10^{-1}	3.162278×10^{-1}	3.162278×10^{-1}	1.01×10^{-9}	0
10^{-2}	10^{-1}	10^{-1}	3.93×10^{-9}	0.000004
10^{-3}	3.162278×10^{-2}	3.162276×10^{-2}	1.29×10^{-8}	0.000041
10^{-4}	10^{-2}	9.949357×10^{-3}	5.06×10^{-5}	0.506425
10^{-5}	3.162278×10^{-3}	1.581076×10^{-3}	1.58×10^{-3}	50.002
10^{-6}	10^{-3}	1.581076×10^{-4}	8.42×10^{-4}	84.189244
10^{-7}	3.162278×10^{-4}	1.581076×10^{-5}	3.00×10^{-4}	95.0002
10^{-8}	10^{-4}	1.581076×10^{-6}	9.84×10^{-5}	98.418924
10^{-9}	3.162278×10^{-5}	1.581076×10^{-7}	3.15×10^{-5}	99.50002
10^{-10}	10^{-5}	1.581076×10^{-8}	9.98×10^{-6}	99.841892

Table 2: Sample error values

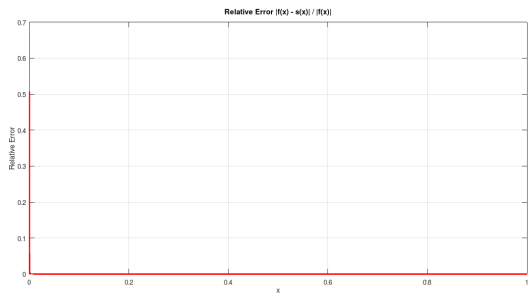


Figure 4: Relative error over the interval

Observations

1. The approximation is accurate away from $x = 0$, but requires a very large $n = 24998$.
2. A sharp increase in error occurs near $x = 0$ due to the unbounded second derivative.
3. The maximum error on $[10^{-4}, 1]$ is close to the prescribed tolerance, validating the theoretical estimate.
4. The error decreases as x increases since $|f''(x)| = \frac{1}{4x^{3/2}}$ decreases.
5. High curvature near $x = 0$ necessitates very fine discretization.
6. Uniform partitioning is inefficient; most intervals are unnecessarily small in smooth regions.
7. A non-uniform (adaptive) partition would significantly reduce n while maintaining accuracy.
8. Higher computational time arises due to the large number of subintervals.

Problem 2

Problem Statement

Write a MATLAB code for piecewise quadratic Lagrange approximation. Compute the approximation to the following functions accurate to within ε

- $f(x) = \sin x$ in $[0, 2\pi]$; $\varepsilon = 5.0 \times 10^{-7}$.
- $f(x) = x^{\frac{1}{2}}$ in $[0, 1]$; $\varepsilon = 5 \times 10^{-4}$.

Show $f(x)$ and its approximation graphically and find the computational time.

Piecewise Quadratic Interpolant and Error Estimates

A quadratic interpolant uses three points. Hence, we partition $[a, b]$ into $2n$ subintervals with $2n + 1$ uniformly spaced nodes

$$x_0, x_1, \ldots, x_{2n}, \quad h = \frac{b-a}{2n}.$$

On each pair of subintervals $[x_{2m-2}, x_{2m}]$, we construct a quadratic interpolant using the nodes $x_{2m-2}, x_{2m-1}, x_{2m}$.

We use the **Newton form** of the interpolating polynomial (since interpolants are unique and this form is convenient computationally):

$$p_m(x) = a_m + b_m(x - x_{2m-2}) + c_m(x - x_{2m-2})(x - x_{2m-1}),$$

where

$$\boxed{a_m = f(x_{2m-2})}, \quad \boxed{b_m = \frac{f(x_{2m-1}) - f(x_{2m-2})}{h}}, \quad \boxed{c_m = \frac{f(x_{2m}) - 2f(x_{2m-1}) + f(x_{2m-2})}{2h^2}}.$$

Thus, the piecewise quadratic interpolant approximates f using parabolas over pairs of subintervals.

Error Estimate: If $f \in C^3[a, b]$, then

$$\max_{x \in [x_{2m-2}, x_{2m}]} |f(x) - p_m(x)| \leq \frac{h^3}{9\sqrt{3}} \max_{x \in [x_{2m-2}, x_{2m}]} |f^{(3)}(x)|.$$

Taking maximum over all intervals,

$$\boxed{\max_{x \in [a,b]} |f(x) - s(x)| \leq \frac{h^3}{9\sqrt{3}} \max_{x \in [a,b]} |f^{(3)}(x)|}$$

Uniform Partition Case: With $h = \frac{b-a}{2n}$,

$$\boxed{|e| \leq \frac{(b-a)^3}{72\sqrt{3}\,n^3} \max_{x\in[a,b]} |f^{(3)}(x)|}$$

To ensure $|e| < \varepsilon$, it suffices that

$$\boxed{n > \frac{b-a}{2}\left(\frac{1}{9\sqrt{3}\,\varepsilon} \max_{x\in[a,b]} |f^{(3)}(x)|\right)^{\frac{1}{3}}}$$

Computation of Required Number of Subintervals

Using the error bound

$$|e| \leq \frac{(b-a)^3}{72\sqrt{3}\,n^3} \max_{x\in[a,b]} |f^{(3)}(x)|,$$

we compute suitable values of n.

- **(a)** $f(x) = \sin x$ on $[0, 2\pi]$, with $\varepsilon = 5 \times 10^{-7}$

$$f^{(3)}(x) = -\cos x, \quad \max |f^{(3)}(x)| = 1.$$

$$n^3 > \frac{(2\pi)^3}{72\sqrt{3}\,\varepsilon} \;\Rightarrow\; n > 158.4.$$

$$\boxed{n = 159}$$

- **(b)** $f(x) = \sqrt{x}$ on $[0, 1]$, with $\varepsilon = 5 \times 10^{-4}$

$$f^{(3)}(x) = \frac{3}{8}x^{-5/2},$$

which is unbounded at $x = 0$, so we restrict to $[10^{-4}, 1]$.

$$\max |f^{(3)}(x)| = 3.75 \times 10^9.$$

$$n^3 > \frac{(1-10^{-4})^3}{72\sqrt{3}\,\varepsilon} \cdot 3.75 \times 10^9 \;\Rightarrow\; n > 3915.6.$$

$$\boxed{n = 3916}$$

Note: Accuracy is guaranteed only on $[10^{-4}, 1]$; error near $x = 0$ may be large.

MATLAB code for Piecewise Quadratic Interpolation

We use the following function to build and plot **Piecewise Quadratic Approximations** :

```
function [] = Problem_2(f, a, b, n)
    tic;

    h = (b - a) / (2*n); % Step Size

    % Choosing Uniformly Spaces 2*n+1 points in the interval [a,b]
    x = linspace(a, b, 2*n+1);

    % Calculate functional values at these points
    fi = f(x);

    ai = zeros(1, n);
    for i = 1 : n
        ai(i) = fi(2*i-1);
    end
    % Initializing array for bi's
    bi = zeros(1, n);

    % Calculating bi's
    for i = 1 : n
        bi(i) = (fi(2*i) - fi(2*i-1)) / h;
    end

    ci = zeros(1, n);

    % Calculating ci's
    for i = 1 : n
        ci(i) = (fi(2*i+1) - 2*fi(2*i) + fi(2*i-1)) / (2*h^2);
    end
    toc;

    % Plotting
    figure;
    hold on;

    % Choosing Dense points (say 10000)
    test_points = linspace(a, b, 10000);
    functional_value = f(test_points);

    % Approx values
```

```
    approx_value = zeros(size(test_points));

    for j = 1:length(test_points)

        % Finding the subinterval
        i = floor((test_points(j) - a)/(2*h)) + 1;
        if i > n
            i = n;
        end
        x0 = x(2*i - 1);
        % Quadratic interpolation
        x1 = x(2*i);
        approx_value(j) = ai(i) ...
            + bi(i)*(test_points(j) - x0) ...
            + ci(i)*(test_points(j) - x0)*(test_points(j) - x1);
    end

    % Plot comparison
    plot(test_points, functional_value, 'b', 'LineWidth', 2);
    plot(test_points, approx_value, 'r--', 'LineWidth', 2);

    legend('Actual f(x)', 'Piecewise Quadratic Approx');
    title('Piecewise Quadratic Interpolation');
    xlabel('x'); ylabel('y');
    grid on;

    % Error
    max_error = max(abs(functional_value - approx_value));
    disp(['Maximum Error = ', num2str(max_error)]);

    % Error plot (pointwise error)
    error_values = abs(functional_value - approx_value);

    figure;
    plot(test_points, error_values, 'k', 'LineWidth', 2);

    title('Pointwise Error |f(x) - s(x)|');
    xlabel('x'); ylabel('Error');
    grid on;

    % Plotting Relative error
    rel_error = (error_values ./ max(abs(functional_value), 1e-12))
       *100;
```

```

    figure;
    plot(test_points, rel_error, 'r', 'LineWidth', 2);

    title('Relative Error |f(x) - s(x)| / |f(x)|');
    xlabel('x'); ylabel('Relative Error');
    grid on;

end
```

Function call parameters for $f(x) = \sin(x)$ are as follows :

```
octave:1> f = @(x) sin(x);
octave:2> a = 0;
octave:3> b = 2*pi;
octave:4> n = 159;
octave:5> Problem_2(f, a, b, n);
```

Function call parameters for $f(x) = x^{\frac{1}{2}}$ are as follows :

```
octave:1> f = @(x) sqrt(x);
octave:2> a = 0;
octave:3> b = 1;
octave:4> n = 3916;
octave:5> Problem_2(f, a, b, n);
```

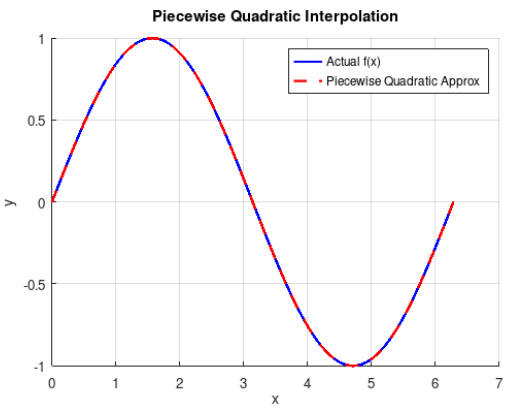
Results for $f(x) = \sin(x)$

For $f(x) = \sin(x)$, the numerical results are:

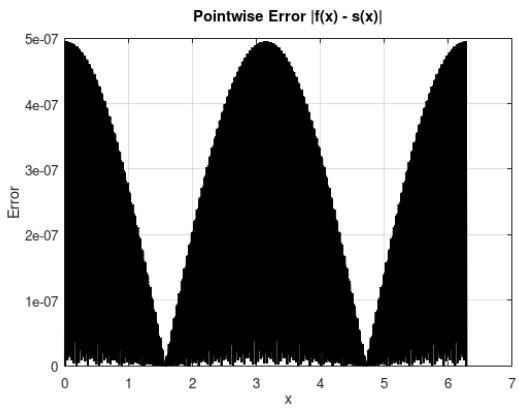
```
Elapsed time is 0.00722504 seconds.
Maximum Error = 4.9456e-07
```

The elapsed time excludes plot generation.

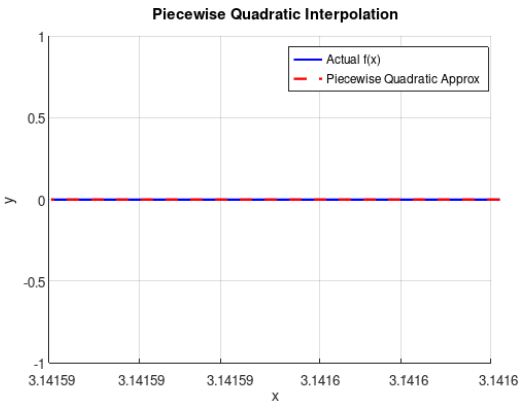
Plots



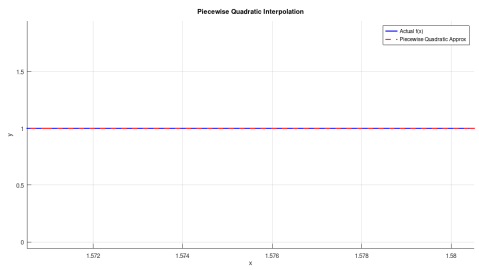
(a) Overall approximation on $[0, 2\pi]$



(b) Pointwise error



(c) Behaviour near $x = \pi$



(d) Behaviour near $x = \pi/2$

Figure 5: Approximation and error visualization

Error Analysis

x	Actual	Approx	Abs Error	Rel Error (%)
0	0	0	0	0
0.6283	0.587785	0.587785	3.02×10^{-7}	5.14×10^{-5}
1.2566	0.951057	0.951056	1.57×10^{-7}	1.65×10^{-5}
1.8850	0.951057	0.951057	1.31×10^{-7}	1.38×10^{-5}
2.5133	0.587785	0.587785	1.99×10^{-7}	3.39×10^{-5}
π	1.22×10^{-16}	2.12×10^{-16}	8.92×10^{-17}	8.92×10^{-3}
3.7699	-0.587785	-0.587785	1.99×10^{-7}	3.39×10^{-5}
4.3982	-0.951057	-0.951057	1.31×10^{-7}	1.38×10^{-5}
5.0265	-0.951057	-0.951056	1.57×10^{-7}	1.65×10^{-5}
5.6549	-0.587785	-0.587785	3.02×10^{-7}	5.14×10^{-5}
2π	-2.45×10^{-16}	-4.23×10^{-16}	1.78×10^{-16}	1.78×10^{-2}

Table 3: Error analysis for $\sin x$

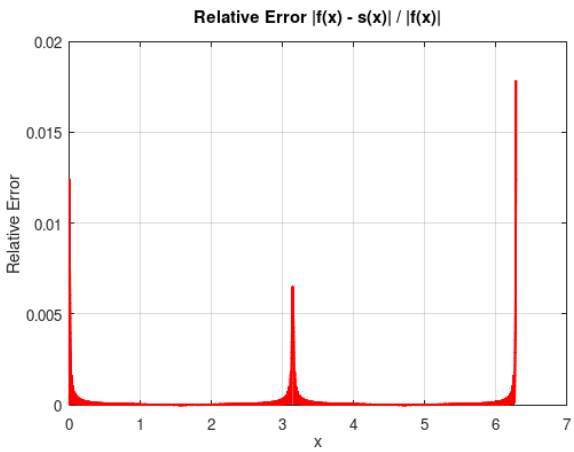


Figure 6: Relative error over $[0, 2\pi]$

Observations

1. The approximation is significantly more accurate than piecewise linear interpolation, since quadratic polynomials capture curvature directly.

2. The error behaves like
$$|f(x) - s(x)| \leq C\, h^3 \max |f^{(3)}(x)|,$$
which explains the much smaller error for comparable n.

3. Since $f^{(3)}(x) = -\cos x$ is bounded, the interpolation is uniformly stable across the interval $[0, 2\pi]$.

4. High-curvature regions (such as near $x = \frac{\pi}{2}$ and $x = \frac{3\pi}{2}$) are handled much better compared to linear interpolation.

5. The error distribution is more uniform across the domain, with no sharp spikes.

6. The symmetry of $\sin x$ is better preserved due to the higher-order approximation.

7. A much smaller number of subintervals is sufficient to achieve the same accuracy compared to linear interpolation.

8. Although the computational cost per interval increases slightly, the overall efficiency improves due to the reduced number of intervals.

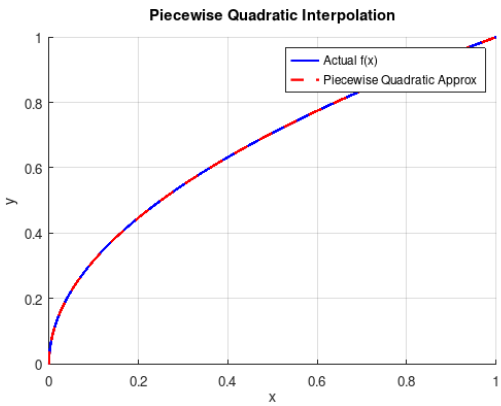
Results for $f(x) = x^{\frac{1}{2}}$

For $f(x) = x^{\frac{1}{2}}$, the numerical results are:

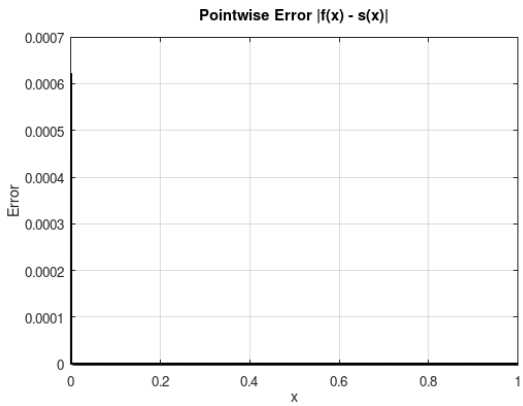
```
Elapsed time is 0.158514 seconds.
Maximum Error = 0.00062125
```

The elapsed time excludes plot generation.

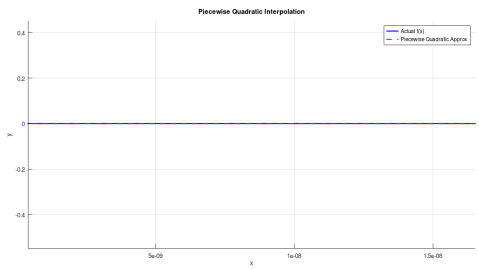
Plots



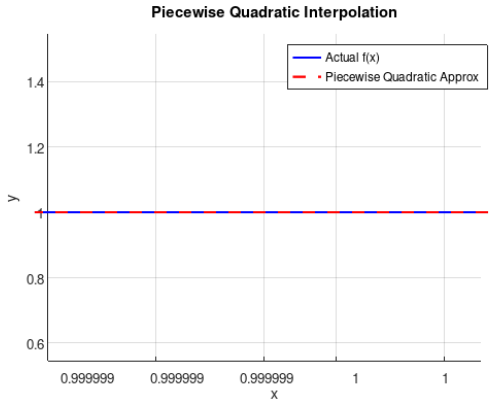
(a) Overall approximation on $[10^{-4}, 1]$



(b) Pointwise error



(c) Behaviour near $x = 0$



(d) Behaviour near $x = 1$

Figure 7: Approximation and error visualization

Error Analysis

x	Actual	Approx	Abs Error	Rel Error (%)
10^{-10}	10^{-5}	1.14×10^{-8}	9.99×10^{-6}	9.99×10^{1}
10^{-9}	3.16×10^{-5}	1.14×10^{-7}	3.15×10^{-5}	9.96×10^{1}
10^{-8}	10^{-4}	1.14×10^{-6}	9.89×10^{-5}	9.89×10^{1}
10^{-7}	3.16×10^{-4}	1.14×10^{-5}	3.05×10^{-4}	9.64×10^{1}
10^{-5}	3.16×10^{-3}	1.12×10^{-3}	2.04×10^{-3}	6.45×10^{1}
10^{-3}	3.16×10^{-2}	3.16×10^{-2}	1.31×10^{-6}	4.15×10^{-3}
10^{-2}	10^{-1}	9.9999995×10^{-2}	4.68×10^{-9}	4.68×10^{-6}
10^{-1}	3.16×10^{-1}	3.16×10^{-1}	7.90×10^{-12}	2.50×10^{-9}
0.9	9.49×10^{-1}	9.49×10^{-1}	3.25×10^{-14}	3.43×10^{-12}
1	1	1	0	0

Table 4: Error behavior near $x = 0$ for $\sqrt{x}$ interpolation

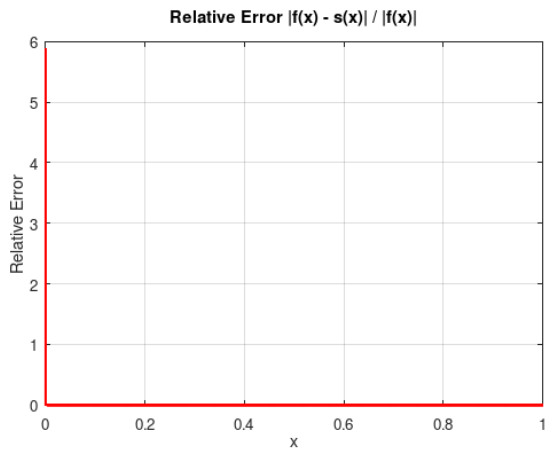


Figure 8: Relative error over the interval

Observations

1. The approximation improves over linear interpolation but still struggles near $x = 0$.
2. The theoretical error depends on

$$f^{(3)}(x) = -\frac{3}{8x^{5/2}},$$

   which becomes unbounded as $x \to 0$, leading to large local errors.
3. Quadratic interpolation reduces the error away from zero, but near $x = 0$, the improvement is limited due to the singular behavior.
4. Compared to linear interpolation, the required number of subintervals is smaller, but still large due to the high curvature near $x = 0$.
5. The error decreases rapidly as x increases since higher derivatives diminish.
6. Uniform partitioning is inefficient, as most intervals in smoother regions (near $x = 1$) are unnecessarily small.
7. Quadratic interpolation partially mitigates curvature effects but does not eliminate the need for adaptive refinement.
8. Relative error near $x = 0$ remains large because the function values themselves are very small.
9. Computational cost is reduced compared to linear interpolation for the same tolerance, but remains high due to dense discretization near $x = 0$.

Problem 3

Problem Statement

Write a numerical code for adaptive approximation (use linear Lagrange interpolating polynomials). Compute the approximation to $f(x) = x^{1/2}$ in $[0, 1]$ accurate to within $\varepsilon = 0.00005$ and $\varepsilon = 5.0 \times 10^{-8}$. Compare the computational time of this algorithm with the computational time of piecewise linear approximation for $\varepsilon = 0.00005$.

Designing an Algorithm for Adaptive Linear Approximation

Let f be the function we want to approximate and let ε be the required tolerance. We want to construct a piecewise linear approximation $s(x)$. We construct a partition $\mathcal{P} = \{x_0, x_1, \ldots, x_n\}$ of $[a, b]$ recursively as follows.

For any subinterval $[x_i, x_{i+1}]$, let the midpoint be

$$x_m = \frac{x_i + x_{i+1}}{2}.$$

Let $L(x)$ denote the linear interpolant on $[x_i, x_{i+1}]$. We have already established the maximum theoretical error bound as

$$|e| \leq \frac{1}{8} h^2 \max_{x \in [x_i, x_{i+1}]} |f''(x)|$$

For the given function $f(x) = \sqrt{x}$, we have

$$f''(x) = -\frac{1}{4} x^{-3/2}.$$

Hence for $x \in [x_i, x_{i+1}]$,

$$\max_{x \in [x_i, x_{i+1}]} |f''(x)| = \frac{1}{4} x_i^{-3/2},$$

since $|f''(x)|$ is decreasing for $x > 0$, the theoretical error bound becomes

$$|e| \leq \frac{1}{8} h^2 \cdot \frac{1}{4} x_i^{-3/2} = \frac{(x_{i+1} - x_i)^2}{32\, x_i^{3/2}}.$$

Adaptive Splitting Criterion: The following recursive adaptive splitting algorithm is used to determine whether to split any subinterval $[a, b]$ into further subintervals or not.

- For intervals sufficiently away from zero ($a \geq 10^{-12}$ say), we use the theoretical bound:

$$e \approx \frac{(b - a)^2}{32\, a^{3/2}}.$$

- Near $x = 0$, the bound blows up due to unbounded-ness of $f''(x)$. Hence, we can use the

error at the mid point as a practical error estimator for such intervals:

$$e \approx |f(m) - L(m)|, \quad m = \frac{a+b}{2}.$$

For each interval $[a, b]$, if the error e is within the tolerance the interval is accepted otherwise it is divided further.

Implementation Details

The function **partition** is the entry function for constructing the adaptive partition of the interval $[a, b]$. It initializes the partition and calls a recursive procedure to refine the interval based on the given tolerance.
The core part of the algorithm is implemented in **partition_rec**, which performs recursive interval subdivision. For each subinterval, the function evaluates the quality of linear approximation using the error estimator discussed above. Depending on the interval, either a theoretical error bound or a midpoint-based estimate for the quality of partition is used. If the estimated error satisfies the tolerance, the interval is accepted, otherwise, it is divided further.
The function **approximate** constructs the piecewise linear interpolant over the generated partition. Given a set of points, it identifies the corresponding subinterval for each point and finds the approximate value using the function values at the endpoints of that subinterval.
The **main script** is used to call all the other functions and to generate plots.

MATLAB Implementations for all Required Functions

Below are the implementations of all the functions listed in the previous section :

1.
```
% Returns adaptive partition of [a,b]
function X = partition(a, b, tol)
    X = a; % start with left endpoint
    X = [X, partition_rec(a, b, tol)];
end
```

2.
```
% Performs the recursive interval splitting
function X = partition_rec(a, b, tol)
    % Evaluating the function at both end points and middle point
    fa = sqrt(a);
    fb = sqrt(b);

    m = (a + b)/2;
    fm = sqrt(m);

    % Finding the linear approximation at middle point
    Lm = fa + (fb - fa)*(m - a)/(b - a);

    % Estiamting quality of the partition
    % if a is large enough we use the theoretical maximum
    % if a too close to zero we take the error at mid point as a
       measaure of quality of approximation
    if a < 1e-12
        err = abs(fm - Lm);
    else
        err = (b - a)^2 / (32 * a^(3/2));
    end

    % if required tolerance is achieved we return the partition
    if err <= tol
        X = b;
    % if not then we recursively call the function for the left
       half and right half sub intervals
    else
        X_left = partition_rec(a, m, tol);
        X_right = partition_rec(m, b, tol);
        X = [X_left, X_right];
    end
end
```

3.

```
% Evaluates the approximation at some query points (xq)
function yq = approximate(X, f, xq)
    % Evaluate piecewise linear interpolation

    Y = f(X); % function values at nodes
    n = length(X);

    yq = zeros(size(xq)); % Array to store approximations

    for k = 1:length(xq)
        x = xq(k);

        % Find interval in which x lies
        i = find(X <= x, 1, 'last');

        % Handle right boundary
        if i == n
            i = n - 1;
        end

        % Endpoints
        x1 = X(i); x2 = X(i+1);
        y1 = Y(i); y2 = Y(i+1);

        % Linear interpolation
        yq(k) = y1 + (y2 - y1)*(x - x1)/(x2 - x1);
    end
end
```

4.

```
% Main Script For the Adaptive Approximation and Plotting
clc; clear; close all;
format long;
f = @(x) sqrt(x);
tol = 1e-5;
a = 0;
b = 1;
%% Adaptive method
tic;
X = partition(a, b, tol);
toc

disp(["Number of Partitions: " num2str(length(X)-1)])
```

```
xq = linspace(a,b,1000);

% Plotting
figure;
hold on;

% Choose Dense points for plotting
test_points = linspace(a, b, 10000);
functional_value = f(test_points);

% Use the generated partition
approx_value = approximate(X, f, test_points);

% Plot comparison
plot(test_points, functional_value, 'b', 'LineWidth', 2);
plot(test_points, approx_value, 'r--', 'LineWidth', 2);

legend('Actual f(x)', 'Piecewise Linear Approx');
title('Adaptive Piecewise Linear Interpolation');
xlabel('x'); ylabel('y');
grid on;

% Error
error_values = abs(functional_value - approx_value);
max_error = max(error_values);

disp(['Maximum Error = ', num2str(max_error)]);

% Error plot
figure;
plot(test_points, error_values, 'k', 'LineWidth', 2);

title('Pointwise Error |f(x) - s(x)|');
xlabel('x'); ylabel('Error');
grid on;

% Relative error
rel_error = (error_values ./ max(abs(functional_value), 1e-12))
    *100;

figure;
plot(test_points, rel_error, 'r', 'LineWidth', 2);
```

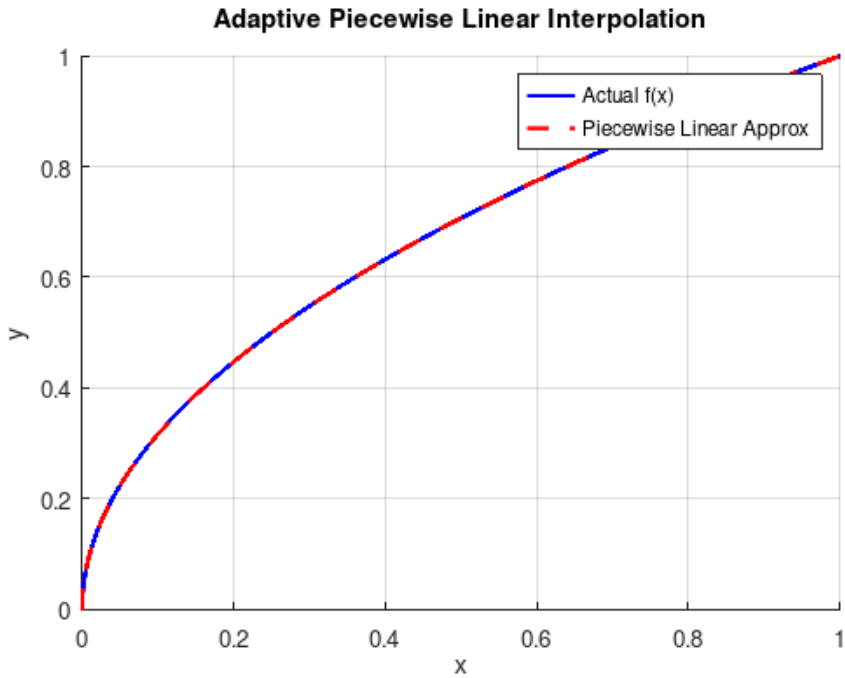
```
title('Relative Error |f(x) - s(x)| / |f(x)|');
xlabel('x'); ylabel('Relative Error (%)');
grid on;
```

Results and Comparison with Other Methods

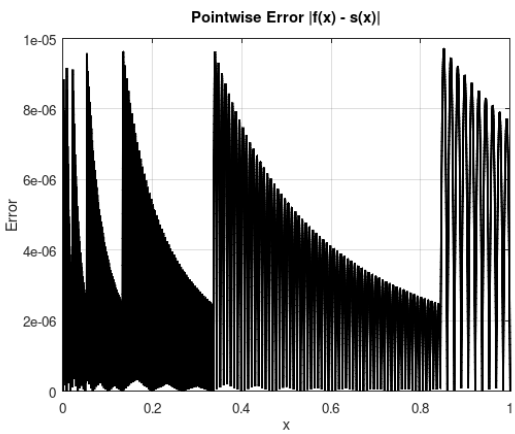
Results and Plots

On running the main script the following output and plots are obtained :

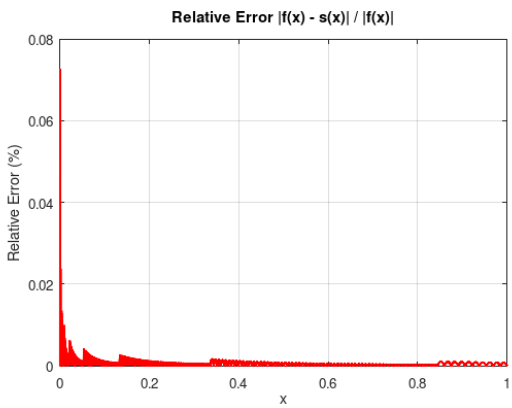
```
Elapsed time is 0.0647209 seconds.
Number of Partitions: 334
Maximum Error = 9.7088e-06
```



(a) Overall approximation in $[0, 1]$



(b) Pointwise Error Plot



(c) Relative Error Plot

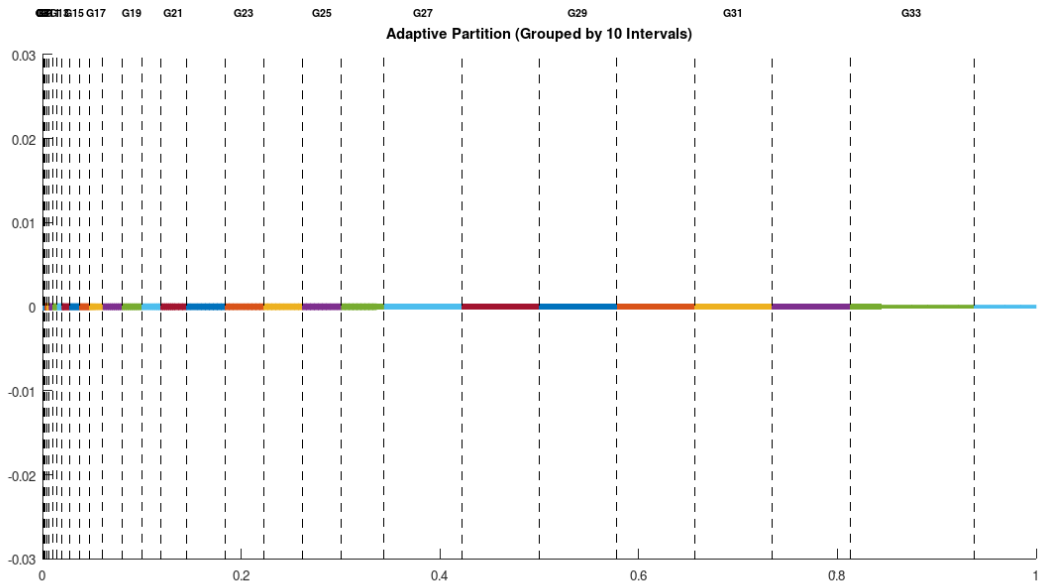


Figure 10: Obtained Adaptive Partition

The final partition of $[0, 1]$ generated by this adaptive approximation scheme showed in the plot above.

Comparison with Piecewise Linear and Quadratic Approximations

The comparison of the three methods are presented here.

Method	Computational Time (s)	No. of Intervals	Max Error
Piecewise Linear	0.312404	24998	5.064×10^{-5}
Piecewise Quadratic	0.158514	3916	6.2×10^{-5}
Adaptive Linear	0.0647209	334	9.708×10^{-6}

Table 5: Comparison of approximation methods

Observations

From the above Table many observations can be made about the performance of the three approximation methods.

1. **Accuracy:** The adaptive linear method achieves the lowest maximum error ($\approx 9.7 \times 10^{-6}$). This indicates that adaptivity significantly improves accuracy for a fixed tolerance.

2. **Computational Efficiency:** The adaptive method is also the fastest among the three, requiring approximately 0.065 seconds. On the other hand, the uniform piecewise linear method is the most computationally expensive due to the large number of intervals.

3. **Number of Intervals:** A huge reduction in the number of intervals is observed for the adaptive method (334 intervals) compared to piecewise linear (24998 intervals) and piecewise quadratic (3916 intervals).

4. **Effect of Function Behavior:** The adaptive partition is denser near $x = 0$, where $f(x) = \sqrt{x}$ has unbounded second derivative, and coarser elsewhere. This confirms that the algorithm effectively captures regions of such high curvatures.

Overall, the adaptive linear approximation provides the best balance between accuracy, computational cost, and number of intervals, making it the most efficient method among those considered.